THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

AAE4011: Artificial Intelligence in Unmanned Autonomous Systems

Project Report

Topic: AI-Powered Semantic Navigation for Indoor Mobile Robots

Dr. Weisong Wen

Group 10

Prithula Prashun Aishy (22098865D)
Wu Chun Him (25016333D)

Table of Contents

Introduction 2

Methodology 2

Implementation 5

Results 9

Introduction

Our project is to work on indoor mobile robots. Many companies are now using it to offer services such as Oliver's Super Sandwiches, which uses robots to deliver food to customers. Safety is important for robots to navigate a correct path. Therefore, they need to understand scene context, identify hazardous areas, interpret visual scenes and generate semantically safe paths. In this project, our system identifies 4 key navigation classes from any indoor image and instead of relying solely on raw images or geometric obstacle detection, the system understands the semantic context of the scene to generate a semantically safe and optimal path.

Methodology

1. Semantic Segmentation Model

A U-Net architecture with a ResNet50 encoder was employed for pixel-wise semantic segmentation. Transfer learning was applied using ImageNet-pretrained weights (1.4 million images), enabling the model to recognize low-level features such as edges, textures, and common objects. The encoder-decoder structure with skip connections preserves spatial information, producing high-resolution output masks.

The model classifies each pixel into one of four navigation-relevant classes: Class
0 – Floor: Safe to traverse (e.g., floor, carpet, rug, path)
Class 1 – Wall/Obstacle: Blocked (e.g., walls, furniture, cabinets) Class
2 – Door: Passable transition areas
Class 3 – No-go zones: Restricted areas (e.g., stairs, fragile surfaces)

2. Dataset and Preprocessing

The ADE20K dataset was used, containing 20,210 training images and 2,000 validation images of diverse indoor scenes. A custom label mapper was implemented to remap the original 150 ADE20K classes into the four navigation classes. The mapping was stored as a Python dictionary (pkl file) and applied to each annotation mask before training.

A custom PyTorch IndoorNavigationDataset class was developed to load matched image–mask pairs. Each sample underwent:

- Loading RGB images and grayscale masks using OpenCV
- Resizing both image and mask to 256×256 pixels (nearest-neighbor interpolation for masks to preserve class labels)
- Remapping class IDs from 1–150 to 0–3
- Application of data augmentation and conversion to tensors

3. Data Augmentation

To improve model generalization and invariance to real-world variations, the following argumentations were applied randomly:

- Horizontal flips
- Rotation (±15°)
- Brightness and contrast jitter

These techniques synthetically increased the effective training set size and reduced overfitting to specific image orientations or lighting conditions.

4. Training Configuration

The model was trained using the following hyperparameters:

- Batch size: 16
- Initial learning rate: 1×10^{-4}
- Epochs: 40
- Optimizer: AdamW with weight decay
- Loss function: Weighted cross-entropy (to handle class imbalance)

A ReduceLROnPlateau scheduler monitored validation mean Intersection over Union (mIoU) after every epoch. If mIoU did not improve for four consecutive epochs (patience=4), the learning rate was halved (factor=0.5). The best model was saved based on validation loss.

The final validation loss achieved was 0.3663.

5. Cost Map Generation

The trained segmentation model produces a class ID per pixel, which is then converted into a numerical cost map — a 2D grid where each cell represents the traversal expense for the robot. The process_image() method executes the following pipeline:

1. Convert input BGR image to RGB
2. Run inference to obtain segmentation mask
3. Generate a colour-coded visual mask for human inspection
4. Create the cost map using predefined class costs (e.g., floor → low cost, obstacle → infinite cost)
5. Produce a colour-coded cost map visualisation for debugging

Outputs are saved as:

cost_map.npy (raw float64 array)

colored_segmentation.png

cost_viz.png

6. Path Planning with Safety Margin

A hybrid path planning approach was implemented combining Probabilistic Roadmap (PRM) and A* algorithms.

PRM Graph Construction:

Free space nodes were sampled and connected to their nearest neighbours to form a roadmap. Bresenham's line algorithm was used to compute approximate straight-line connections between nodes using only integer arithmetic, ensuring computational efficiency.

Obstacle Inflation:

To maintain a safety margin, obstacle boundaries were expanded by a defined number of pixels. This prevents the robot from traveling too close to walls or blocked regions.

*A Search:**

Given the cost map, the A* algorithm finds the lowest-cost path from a start to a goal position. Blocked cells (class 1 or 3) were assigned at an infinite cost. Additionally, a distance transform was applied to compute each pixel's proximity to the nearest obstacle, and a safety penalty was added to nearby cells. The final path preferentially selects traversable areas, doors and safe distance from walls.

7. Implementation Tools

The project was implemented in Python using PyTorch for deep learning, OpenCV for image processing, and custom modules for cost mapping and path planning. All experiments were conducted in a standard computing environment with GPU acceleration.

Implementation.

1. Model Architecture and Training

A **U-Net architecture with a ResNet50 encoder** was employed for pixel-wise semantic segmentation. The model outputs raw logits for 4 classes, uses batch normalization in the decoder, and includes a dropout rate of 0.3 for regularization.

```python
class IndoorSegmentationModel(nn.Module):
    """
    U-Net with ResNet50 backbone for indoor scene segmentation.
    Outputs 4 classes: floor, obstacle/wall, door, no-go.
    """

    def __init__(self, num_classes=None, encoder_name=None, encoder_weights=None):
        super(IndoorSegmentationModel, self).__init__()

        self.num_classes    = num_classes    or getattr(Config, 'NUM_CLASSES', 4)
        self.encoder_name   = encoder_name   or getattr(Config, 'ENCODER', 'resnet50')
        encoder_weights     = encoder_weights or getattr(Config, 'ENCODER_WEIGHTS', 'imagenet')

        # U-Net with pre-trained ResNet50 encoder
        self.model = smp.Unet(
            encoder_name=self.encoder_name,
            encoder_weights=encoder_weights,
            in_channels=3,
            classes=self.num_classes,
            activation=None,            # Raw logits — required for CrossEntropyLoss
            decoder_use_batchnorm=True,
            decoder_dropout=0.3,        # ← consistent with train.py
        )
```

Figure 1

Training Configuration: The training script initializes the configuration, sets up the

device and deletes any existing checkpoint for a fresh start.

```python
def main():
    Config.setup()

    device = Config.get_device()
    print(f"\n🔧 Detected device: {device}")
    if device == 'cpu':
        print("⚠️ WARNING: Training on CPU — this will be very slow!")
    else:
        print(f"   GPU: {torch.cuda.get_device_name(0)}")

    Config.print_config()

    # ── Fresh start — delete old checkpoint ──
    checkpoint_path = os.path.join(Config.MODEL_SAVE_PATH, 'best_model.pth')
    if os.path.exists(checkpoint_path):
        os.remove(checkpoint_path)
        print("\n🗑️  Deleted old checkpoint — fresh start!")
    else:
        print("\n🚀 No checkpoint found — starting fresh.")
```

Figure 2

2. Inference and Cost Map Generation

The ColorCodedCostMapGenerator class in cost_map_genarator.py organizes the complete inference pipeline. It loads the trained model, performs segmentation on an input image, and generates both a color-coded visualization and a numerical cost map for path planning.

The predict method resizes the input image, normalizes it, and runs inference through the U-Net model to produce a class ID per pixel. The segmentation mask is then resized back to the original image dimensions using nearest-neighbor interpolation to preserve class label integrity.

```python
    def predict(self, image):
        """
        Predict segmentation mask for an image.

        Args:
            image: Input image (numpy array, BGR format from cv2.imread)

        Returns:
            segmentation: 2D array with class IDs (0-3)
        """
        img_resized = cv2.resize(image, (256, 256))

        # Convert BGR → RGB before feeding to model
        img_rgb        = cv2.cvtColor(img_resized, cv2.COLOR_BGR2RGB)
        img_normalized = img_rgb.astype(np.float32) / 255.0
        img_tensor     = torch.from_numpy(img_normalized).permute(2, 0, 1).float(
        img_tensor     = img_tensor.unsqueeze(0).to(self.device)

        with torch.no_grad():
            output = self.model(img_tensor)
            pred   = torch.argmax(output, dim=1).squeeze(0).cpu().numpy()

        # Resize back to original dimensions
        h, w         = image.shape[:2]
        pred_full    = cv2.resize(pred.astype(np.uint8), (w, h),
                                  interpolation=cv2.INTER_NEAREST)
        return pred_full
```

Figure 3

3. Path Planning with A* and Safety Margins

The SafetyAStarPlanner class implements an A* path planner with a safety margin from obstacles. It uses distance transform to compute the distance from every cell to the nearest obstacle and applies a linearly decreasing penalty for cells within the safety margin.

```python
class SafetyAStarPlanner:
    def __init__(self, cost_map, safety_margin=25, obstacle_weight=5.0):
        """
        Initialize the safety-aware planner.

        Args:
            cost_map      : Cost map from segmentation (float64:
                            1.0=floor, 0.8=door, inf=blocked)
            safety_margin : Minimum pixels to keep from obstacles (default: 25)
            obstacle_weight: Extra cost multiplier for cells near obstacles (default: 5.0)
        """
        self.base_cost_map   = cost_map.astype(np.float64)   # FIX 4: enforce float64
        self.height, self.width = cost_map.shape
        self.safety_margin   = safety_margin
        self.obstacle_weight = obstacle_weight

        # FIX 1: use np.isinf() instead of == float('inf')
        self.obstacle_mask = np.isinf(self.base_cost_map)

        # Distance from every cell to nearest obstacle
        self.distance_map = distance_transform_edt(~self.obstacle_mask)

        # Safety-enhanced cost map and traversable mask
        self.cost_map        = self._create_safety_cost_map()
        self.traversable_mask = ~np.isinf(self.cost_map)     # FIX 1

```

Figure 4

Safety Cost Computation: The _create_safety_cost_map method adds penalties near obstacles in a fully vectorized manner. Cells within the safety margin receive a penalty that decreases linearly with distance from the obstacle, encouraging the planner to maintain a safe clearance.

```python
    def _create_safety_cost_map(self):
        """Add safety penalties near obstacles — fully vectorized (FIX 2)."""
        safety_cost = np.zeros_like(self.base_cost_map, dtype=np.float64)  # FIX 4

        # FIX 2: replace pixel-by-pixel loop with vectorized operation
        near_obstacle = (~self.obstacle_mask) & (self.distance_map < self.safety_margin)
        safety_cost[near_obstacle] = self.obstacle_weight * (
            1.0 - self.distance_map[near_obstacle] / self.safety_margin
        )

        final_cost = self.base_cost_map.copy()
        passable   = ~np.isinf(final_cost)                   # FIX 1
        final_cost[passable] += safety_cost[passable]
        return final_cost
```

Figure 5

Results

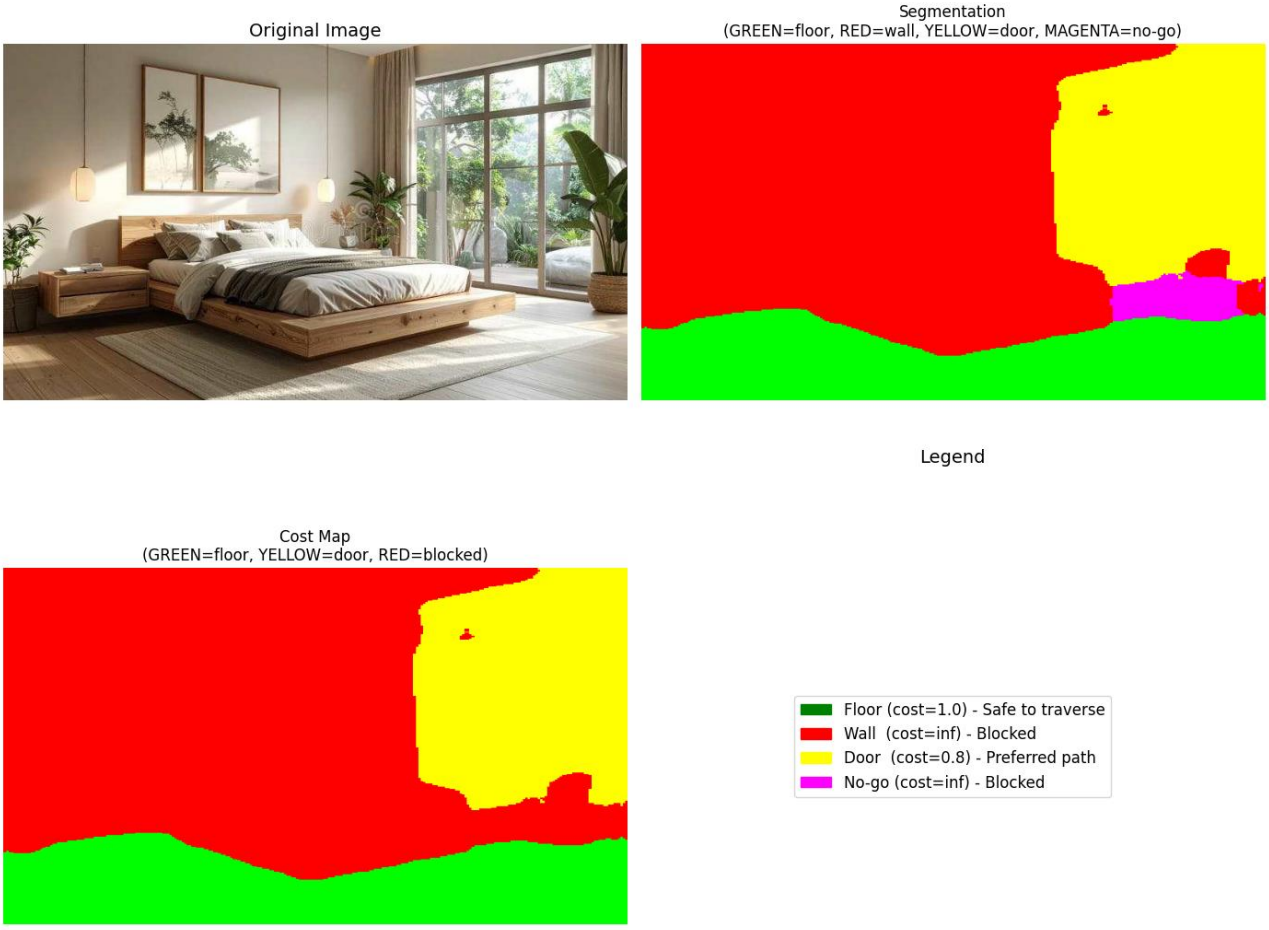

Figure 6

The above picture is the result of image segmentation. The colored photo is the cost map of original image. Different colors represent different obstacles and the cost such as red is wall or yellow is door. Also, they will be shown whether they can be the suitable path.

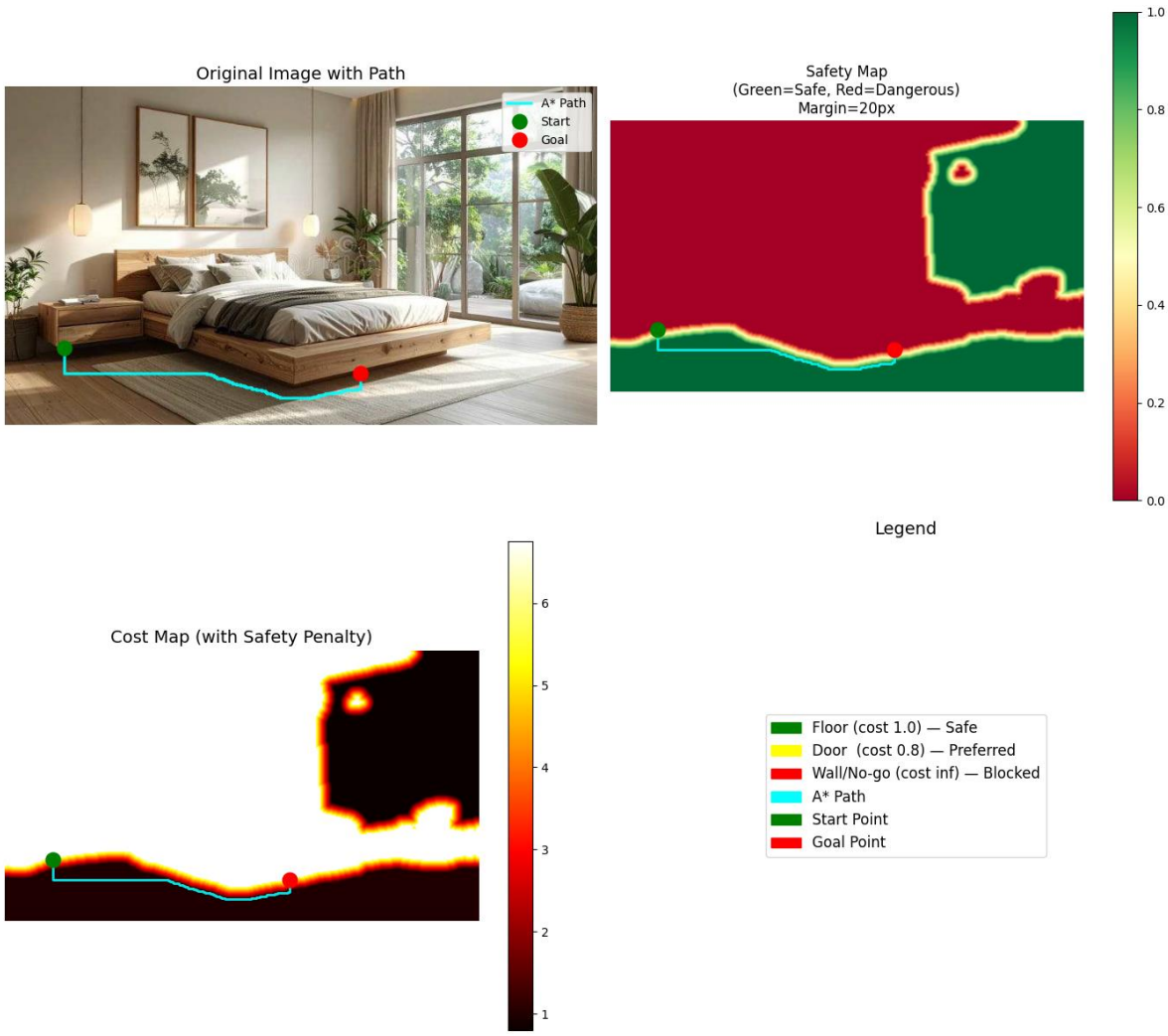

Figure 7

The above picture is the result of path planning. Combining the image segmentation part, it generates a suitable for robot using A* planning algorithm. If the robot detects the wall which is colored red in cost map, it will avoid this obstacle. By using Bresenham's line algorithm, the system will compute a suitable and safety path for robot.

In conclusion, this project demonstrates a robust semantic navigation system for indoor robots. We successfully trained a U-Net model to identify safe paths, obstacles, doors, and no-go zones from raw images. By translating this semantic understanding into a numerical cost map, our A* path planner generates optimal, collision-free routes with built-in safety margins. The results show that combining deep learning scene-parsing with traditional path planning allows robots to navigate complex indoor environments more safely and intelligently.